Contributing

Setup

```
git clone <repo>
cd colab-agent
pip install -e .
```

Code Style

- Python 3.12+, type hints required
- Ruff linting: `ruff check .`
- Line length: 120
- No trailing whitespace, no unused imports/variables
- Docstrings on all public classes, methods, and module-level functions

Test Conventions

Test file	What it tests	Requires API
`test_plugin.py`	Plugin system + LLM cache (48 tests)	No
`test_self_improvement.py`	Error classification, self-improvement plugin, async core (28 tests)	No
`test_config.py`	Settings validation, env profiles (14 tests)	No
`test_api.py`	FastAPI endpoints, OpenAPI schema (7 tests)	No
`test_safety.py`	Cost tracker, budget, code sanitization (27 tests)	No
`test_health.py`	Health reporter (7 tests)	No
`test_observability.py`	Metrics collector, JSON formatter (11 tests)	No
`test_circuit_breaker.py`	Circuit breaker flow (1 test)	No
`test_operations.py`	HTTP handler + migration (6 tests)	No
`test_edge_cases.py`	Memory, safety, autonomy edge cases (56 tests)	No
`test_security_phase1.py`	Input sanitizer, key encryption (37 tests)	No
`test_cli.py`	CLI commands, serve, budget	No

Test file	What it tests	Requires API
	(27 tests)	
`test_prompts.py`	Prompt templates (21 tests)	No
`test_capabilities_codegen.py`	All 13 capabilities generate valid Python (109+ methods)	No
`test_gh_integration.py`	GitHub client, logger (2 pre-existing env-var failures)	GitHub token
`test_integration.py`	LLM client, config, tokens, memory store	LLM API
`test_e2e_agent.py`	Full agent orchestration loop	LLM API
`test_storage.py`	SQLite job/conversation/queue store	No

Running tests

```
# Quick (no API calls, ~60s)
pytest tests/test_plugin.py tests/test_self_improvement.py tests/test_config.py
-v

# All unit tests (ignores API-dependent)
pytest tests/ --ignore=tests/test_models.py --ignore=tests/test_e2e_agent.py --
ignore=tests/test_integration.py --ignore=tests/test_storage.py -q

# Full suite
pytest tests/ -v --timeout=60

# Single file
pytest tests/test_plugin.py -v -k test_thread_safety
```

Adding a Plugin

1. Create a class inheriting from `agent.plugin.Plugin`
2. Override any of the 9 lifecycle hooks:
    - `before_plan(goal, context) -> (goal, context)`
    - `after_plan(plan, context) -> (plan, context)`
    - `before_step(step, context) -> (step, context)`
    - `after_step(step, result, context) -> (result, context)`
    - `before_code_gen(step, prompt, context) -> (prompt, context)`
    - `after_code_gen(step, code, context) -> (code, context)`
    - `on_error(step, error, context) -> (recovery_action_or_None, context)`
    - `on_summary(summary, context) -> (summary, context)`
    - `on_complete(result, context) -> (result, context)`

3. Register with `@plugin(name=..., version=..., priority=...)` decorator
4. Add tests in `tests/test_plugin.py` or a dedicated test file

Adding a Capability

1. Create `capabilities/your_module.py`
2. Each method that generates Colab code must return valid Python (after stripping shell commands)
3. Shell commands (`!pip`, `!apt`, `!git`) are allowed and stripped during AST validation
4. Avoid `"""` inside `return f"""..."""` — use `'''` for inner docstrings and SQL
5. Add a test function to `tests/test_capabilities_codegen.py`

Common Pitfalls

- **Triple-quote nesting**: Inside `return f"""..."""`, use `'''` for inner triple-quoted strings
- **F-string escaping**: Inside the outer f-string, `{...}` is an expression; use `{{...}}` to produce literal braces
- **SCHEMA interpolation**: Don't `{self.SCHEMA}` directly — inline with `cursor.execute('''...''')`
- **`datetime.utcnow()`**: Use `datetime.now(timezone.utc)` instead
- **Import shadowing**: `github/` shadows PyGithub; use `gh_integration/` instead
- **Inner triple-quote rule**: Inside `return f"""..."""`, always use `'''` for inner docstrings/SQL/generated code to avoid premature outer-string closure
- **F-string escaping rule**: Inside outer `f"""..."""`, use `{{...}}` to produce literal `{...}` in output for variables that only exist in generated code

Async Patterns

The agent provides both sync and async entry points:

```
from agent.core import run_agent, async_run_agent

# Sync (blocks)
result = run_agent("Fine-tune Phi-2")

# Async (awaitable)
result = await async_run_agent("Fine-tune Phi-2")
```

When adding new blocking operations to the agent:

1. Add both sync and async (`await asyncio.to_thread(...)`) versions
2. Keep the sync method as the source of truth

3. Mirror changes in `_execute_step` / `_async_execute_step`

Production Hardening Checklist

When adding a new module, ensure:

- ☐ `threading.Lock` (or `RLock` for re-entrant) on all shared mutable state
- ☐ Input validation with clear error messages (prefer `TypeError` for type errors)
- ☐ Error isolation — one component failure doesn't crash unrelated components
- ☐ Graceful degradation — return safe defaults instead of raising on non-critical errors
- ☐ Resource cleanup — `close()` methods, context managers
- ☐ Structured logging — use `logger.info/error/warning` with contextual fields
- ☐ Type hints on all public methods and module-level functions
- ☐ Docstrings on all public classes and methods
- ☐ Thread safety tests with 20+ concurrent workers
- ☐ Malformed/edge-case input tests

PR Process

1. Ensure `ruff check .` passes (0 errors in new code; pre-existing `test_gh_integration.py` E402 grandfathered)
2. Ensure quick tests pass: `pytest tests/test_plugin.py tests/test_self_improvement.py tests/test_config.py tests/test_api.py -v`
3. Add or update tests for new functionality
4. Update README if adding new config, CLI commands, or modules
5. Document new plugins in the plugin section of README